

PostgreSQL Streaming Replication — Complete Setup Guide

Table of Contents

- 1. [Learning Objectives](#)
- 2. [How Streaming Replication Works](#)
- 3. [Architecture Diagram](#)
- 4. [Prerequisites](#)
- 5. [Step-by-Step Setup: Primary Server](#)
- 6. [Step-by-Step Setup: Standby Server](#)
- 7. [pg_hba.conf Configuration](#)
- 8. [Replication Slots](#)
- 9. [Cascading Replication](#)
- 10. [Monitoring Streaming Replication](#)
- 11. [Common Mistakes](#)
- 12. [Best Practices](#)
- 13. [Performance Considerations](#)
- 14. [Interview Questions](#)
- 15. [Exercises and Solutions](#)

Learning Objectives

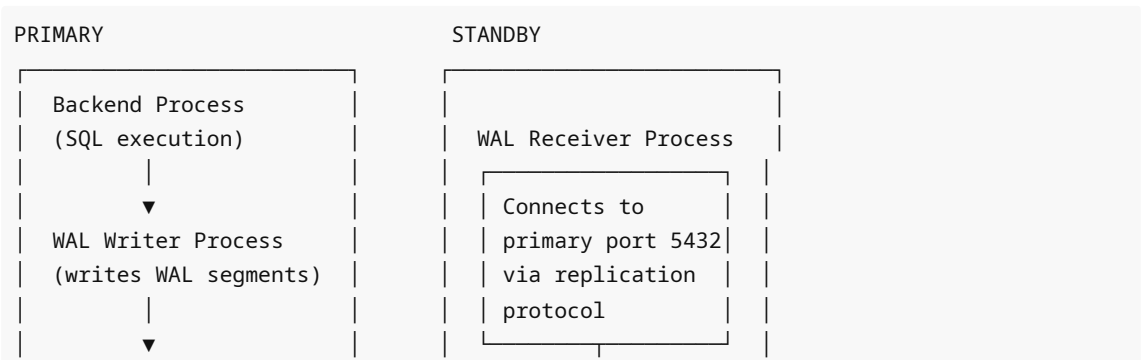
By the end of this module, you will be able to:

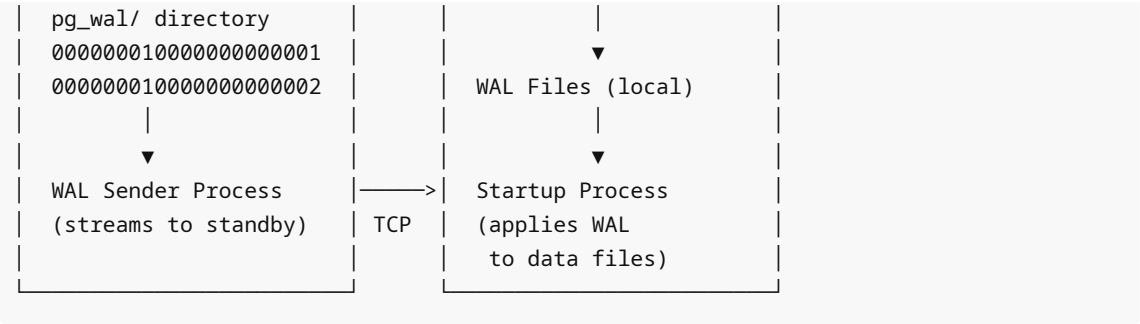
- Configure a full primary/standby streaming replication setup from scratch
- Write correct pg_hba.conf entries for replication connections
- Set up replication slots to prevent WAL deletion
- Monitor replication health and lag
- Handle common streaming replication failures
- Configure hot standby for read queries

How Streaming Replication Works

Streaming replication streams WAL (Write-Ahead Log) records from the primary to standbys in near-real-time over a TCP connection, allowing standbys to stay within seconds of the primary.

Process Architecture





WAL Flow States

A standby connection goes through these states:

- **startup** — Initial handshake and authentication
- **catchup** — Standby is behind and catching up
- **streaming** — Standby is current and streaming in real-time
- **backup** — Used during `pg_basebackup`

Prerequisites

Item	Requirement
PostgreSQL version	Same major version on primary and standby
Network	Primary and standby can reach each other on port 5432
Disk	Standby needs equal or larger disk than primary
OS user	postgres user on both servers
Replication user	A dedicated database role for replication

Step-by-Step Setup: Primary Server

Step 1: Create Replication User

```
-- Connect to primary as superuser
CREATE ROLE replicator WITH REPLICATION LOGIN PASSWORD 'StrongPassword123!';
-- Verify
SELECT rolname, rolreplication, rolcanlogin FROM pg_roles WHERE rolname =
'replicator';
```

Step 2: Configure postgresql.conf on Primary

```
# /etc/postgresql/16/main/postgresql.conf

#-----
# REPLICATION SETTINGS
#-----
```

```

# WAL configuration
wal_level = replica          # Minimum for streaming replication
                              # Use 'logical' if you also need logical
                              replication

# WAL sender processes
max_wal_senders = 10         # Allow up to 10 simultaneous WAL sender
                              connections
                              # Set higher if you have many standbys

# Replication slots
max_replication_slots = 10   # Allow up to 10 replication slots

# WAL retention (safety net without slots)
wal_keep_size = 1GB          # Keep at least 1GB of WAL segments
                              # Prevents standby reconnect failures after brief
                              outage

# WAL compression (saves network bandwidth)
wal_compression = on

# Required for pg_rewind (standby re-sync after divergence)
wal_log_hints = on

# Hot standby conflict resolution
hot_standby_feedback = on    # Standby tells primary about long-running queries
                              # Prevents "canceling statement due to conflict"

# Tracking
track_commit_timestamp = on  # Useful for replication monitoring

#-----
# CONNECTIONS
#-----
listen_addresses = '*'       # Listen on all interfaces (or specific IPs)
port = 5432
max_connections = 200

```

Step 3: Configure pg_hba.conf on Primary

```

# /etc/postgresql/16/main/pg_hba.conf

# TYPE  DATABASE      USER      ADDRESS            METHOD

# Local connections (keep existing entries)
local   all             postgres   peer
local   all             all        md5

# IPv4 local connections
host    all             all        127.0.0.1/32       scram-sha-256

```

```
# Replication connections
# Replace with actual standby IP address(es)
host    replication    replicator    192.168.1.101/32      scram-sha-256
host    replication    replicator    192.168.1.102/32      scram-sha-256
# Or allow entire subnet:
host    replication    replicator    192.168.1.0/24        scram-sha-256

# Application connections
host    all             all           192.168.1.0/24        scram-sha-256
```

Step 4: Reload/Restart Primary

```
# Reload (picks up pg_hba.conf and most postgresql.conf changes)
sudo systemctl reload postgresql@16-main
# or
sudo -u postgres pg_ctl reload -D /var/lib/postgresql/16/main

# Full restart required for wal_level, max_wal_senders, max_replication_slots
sudo systemctl restart postgresql@16-main

# Verify settings took effect
sudo -u postgres psql -c "SHOW wal_level; SHOW max_wal_senders;"
```

Step-by-Step Setup: Standby Server

Step 5: Take a Base Backup

```
# On the STANDBY server, as postgres user
# This copies the entire data directory from primary

# Method 1: pg_basebackup (recommended)
sudo -u postgres pg_basebackup \
  --host=192.168.1.100 \
  --port=5432 \
  --username=replicator \
  --pgdata=/var/lib/postgresql/16/main \
  --wal-method=stream \      # Stream WAL during backup (prevents WAL gaps)
  --checkpoint=fast \       # Don't wait for checkpoint
  --progress \              # Show progress
  --verbose \
  --write-recovery-conf      # Write standby.signal + postgresql.auto.conf

# Enter password when prompted, or use .pgpass file

# Method 2: With replication slot creation
sudo -u postgres pg_basebackup \
  --host=192.168.1.100 \
  --port=5432 \
```

```
--username=replicator \  
--pgdata=/var/lib/postgresql/16/main \  
--wal-method=stream \  
--create-slot \  
--slot=standby1_slot \  
--checkpoint=fast \  
--write-recovery-conf
```

Using .pgpass for Password-Free Authentication

```
# Create .pgpass file on standby (as postgres user)  
echo "192.168.1.100:5432:replication:replicator:StrongPassword123!" \  
>> /var/lib/postgresql/.pgpass  
chmod 600 /var/lib/postgresql/.pgpass  
chown postgres:postgres /var/lib/postgresql/.pgpass
```

Step 6: Configure Standby (postgresql.conf / postgresql.auto.conf)

The `--write-recovery-conf` flag creates a `standby.signal` file and writes to `postgresql.auto.conf`. You can also configure manually:

```
# /var/lib/postgresql/16/main/postgresql.auto.conf  
# (Written by pg_basebackup --write-recovery-conf, or add manually)  
  
primary_conninfo = 'host=192.168.1.100 port=5432 user=replicator  
password=StrongPassword123! application_name=standby1'  
primary_slot_name = 'standby1_slot'    # If using replication slot
```

```
# /etc/postgresql/16/main/postgresql.conf (on standby)  
  
# Hot standby: allow read-only queries  
hot_standby = on                # Allow connections during recovery  
  
# Conflict handling  
hot_standby_feedback = on       # Tell primary about our long queries  
max_standby_streaming_delay = 30s # Cancel conflicting queries after 30s  
max_standby_archive_delay = 30s  
  
# Recovery minimum delay (optional: create a delayed replica)  
# recovery_min_apply_delay = '30min' # Useful for protection against logical errors  
  
# Port (can differ from primary if on same host)  
port = 5432  
  
# Logging  
log_min_messages = info
```

Step 7: Create standby.signal

```
# This file signals PostgreSQL to start in standby/recovery mode
# pg_basebackup --write-recovery-conf creates it automatically
# If not, create it manually:
```

```
sudo -u postgres touch /var/lib/postgresql/16/main/standby.signal
```

```
# Note: In PostgreSQL 12+, recovery.conf is no longer used.
# Settings go in postgresql.conf or postgresql.auto.conf
# and standby.signal triggers standby mode.
```

Step 8: Start the Standby

```
# Start PostgreSQL on standby
sudo systemctl start postgresql@16-main

# Watch the logs to verify it connects
sudo tail -f /var/log/postgresql/postgresql-16-main.log

# Expected log messages:
# LOG:  started streaming WAL from primary at 0/3000000 on timeline 1
# LOG:  redo starts at 0/3000060
# LOG:  consistent recovery state reached at 0/3000130

# Verify standby is in recovery mode
sudo -u postgres psql -c "SELECT pg_is_in_recovery();"
# pg_is_in_recovery
# _____
# t
```

pg_hba.conf Configuration

Authentication Methods for Replication

```
# TRUST: No password (dev/test only, never production)
host    replication    replicator    192.168.1.0/24    trust

# MD5: Older password hash (deprecated in favor of scram-sha-256)
host    replication    replicator    192.168.1.0/24    md5

# SCRAM-SHA-256: Modern password authentication (recommended)
host    replication    replicator    192.168.1.0/24    scram-sha-256

# SSL + SCRAM (most secure)
hostssl replication    replicator    192.168.1.0/24    scram-sha-256

# Certificate authentication (strongest)
hostssl replication    replicator    192.168.1.0/24    cert
```

Verify pg_hba.conf Changes

```
-- View current pg_hba.conf entries
SELECT type, database, user_name, address, auth_method
FROM pg_hba_file_rules
WHERE database = '{replication}';

-- Test replication connection from standby
-- (run on standby host)
-- psql "host=192.168.1.100 port=5432 user=replicator dbname=replication"
```

Replication Slots

Why Use Replication Slots?

Without slots, if a standby falls behind and the primary deletes old WAL segments, the standby cannot reconnect — it needs a full re-sync. Slots guarantee WAL retention until the standby consumes it.

Warning: Unused slots that are never dropped will cause WAL to accumulate indefinitely, potentially filling disk.

Managing Replication Slots

```
-- Create a physical replication slot (on primary)
SELECT pg_create_physical_replication_slot('standby1_slot');

-- Create with WAL immediately (for pg_basebackup)
SELECT pg_create_physical_replication_slot('standby1_slot', true);

-- View all slots
SELECT slot_name, slot_type, active, restart_lsn,
       pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn) AS lag_bytes
FROM pg_replication_slots;

-- Drop a slot (when standby is decommissioned)
SELECT pg_drop_replication_slot('standby1_slot');

-- Monitor slot WAL retention (alert if > 10GB)
SELECT slot_name,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS
wal_behind
FROM pg_replication_slots
WHERE active = false;
```

Slot Configuration in Standby

```
# postgresql.auto.conf on standby
primary_conninfo = 'host=primary port=5432 user=replicator'
```

```
application_name=standby1'  
primary_slot_name = 'standby1_slot'
```

Cascading Replication

A standby can serve as a WAL relay for downstream standbys, reducing the number of WAL sender connections on the primary.

```
PRIMARY —WAL—> STANDBY_1 —WAL—> STANDBY_2  
                  |                |  
            (intermediate)      (downstream)  
            primary_conninfo    primary_conninfo  
            points to PRIMARY   points to STANDBY_1
```

Configuring Cascade

```
# On STANDBY_2 (downstream): postgresql.auto.conf  
primary_conninfo = 'host=192.168.1.101 port=5432 user=replicator  
application_name=standby2'
```

```
# On STANDBY_1 (intermediate): postgresql.conf  
# Must allow WAL senders to downstream replicas  
wal_level = replica  
max_wal_senders = 5  
hot_standby = on
```

```
# On STANDBY_1 (intermediate): pg_hba.conf  
host      replication      replicator      192.168.1.102/32      scram-sha-256
```

Monitoring Streaming Replication

Primary-Side Monitoring

```
-- Full replication status  
SELECT  
    pid,  
    client_addr,  
    application_name,  
    state,  
    sent_lsn,  
    write_lsn,  
    flush_lsn,  
    replay_lsn,  
    write_lag,  
    flush_lag,  
    replay_lag,  
    sync_state,
```



```

        sync_priority
FROM pg_stat_replication;

-- Replication lag in human-readable format
SELECT
    application_name,
    state,
    sync_state,
    replay_lag AS total_lag,
    CASE
        WHEN replay_lag IS NULL THEN 'Not connected'
        WHEN replay_lag < INTERVAL '1 second' THEN 'Healthy'
        WHEN replay_lag < INTERVAL '30 seconds' THEN 'Minor lag'
        WHEN replay_lag < INTERVAL '5 minutes' THEN 'WARNING'
        ELSE 'CRITICAL'
    END AS lag_status
FROM pg_stat_replication;

-- How much WAL has the primary generated
SELECT pg_current_wal_lsn(), pg_current_wal_insert_lsn();

```

Standby-Side Monitoring

```

-- WAL receiver status (run on standby)
SELECT
    status,
    receive_start_lsn,
    receive_start_tli,
    written_lsn,
    flushed_lsn,
    received_tli,
    last_msg_send_time,
    last_msg_receipt_time,
    latest_end_lsn,
    latest_end_time,
    slot_name,
    conninfo
FROM pg_stat_wal_receiver;

-- Confirm standby mode
SELECT pg_is_in_recovery(), pg_last_wal_receive_lsn(), pg_last_wal_replay_lsn();

-- Time since last WAL received
SELECT now() - pg_last_xact_replay_timestamp() AS replication_delay;

```

Shell Monitoring Script

```

#!/bin/bash
# replication_check.sh

```

```
# Run on primary to check all standby lag

PGHOST=localhost
PGUSER=postgres
PGDATABASE=postgres

psql -h $PGHOST -U $PGUSER -d $PGDATABASE -x -c "
SELECT
    application_name,
    client_addr,
    state,
    sync_state,
    write_lag,
    flush_lag,
    replay_lag
FROM pg_stat_replication
ORDER BY replay_lag DESC NULLS LAST;
"

# Alert if any standby has lag > 60 seconds
LAG=$(psql -h $PGHOST -U $PGUSER -d $PGDATABASE -t -c "
SELECT COUNT(*) FROM pg_stat_replication
WHERE replay_lag > INTERVAL '60 seconds';
")

if [ "$LAG" -gt 0 ]; then
    echo "ALERT: $LAG standby(ies) have replication lag > 60 seconds!"
    exit 1
fi
echo "Replication lag OK"
```

Common Mistakes

1. **Wrong wal_level:** Setting `wal_level = minimal` — streaming replication requires at least `replica`
2. **Missing standby.signal:** Forgetting to create `standby.signal` file (PostgreSQL 12+)
3. **Firewall blocking port 5432:** Standby cannot connect to primary
4. **Wrong primary_conninfo:** Typo in host/user/password in `postgresql.auto.conf`
5. **Not using `--wal-method=stream`** with `pg_basebackup` — WAL gap can occur
6. **Leaving slots active after standby decommission** — disk fills with retained WAL
7. **Not setting `hot_standby = on`** — cannot run read queries on standby
8. **Using `recovery.conf` in PostgreSQL 12+** — file is ignored, use `postgresql.conf` + `standby.signal`
9. **Not testing failover** — discovering procedure doesn't work during actual outage
10. **Forgetting to configure `pg_hba.conf`** — replication connections fail auth

Best Practices

1. Use a **dedicated replication user** with only `REPLICATION` privilege
2. Always use **`scram-sha-256`** (not `md5` or `trust`) for replication auth
3. Create **replication slots** for all standbys to prevent WAL deletion

4. Set up **monitoring** for both replication lag and slot WAL accumulation
 5. Test **pg_basebackup** regularly to know how long it takes
 6. Keep **wal_keep_size** as a safety net (1-2GB) even with slots
 7. Enable **wal_compression = on** to save bandwidth
 8. Document and **test failover** procedures at least quarterly
 9. Use **cascading replication** for more than 3-4 standbys
 10. Enable **hot_standby_feedback = on** to reduce standby query cancellations
-

Performance Considerations

WAL Sender Impact

```
# postgresql.conf – tune WAL sender behavior
wal_sender_timeout = 60s      # Disconnect slow standbys after 60s
wal_receiver_timeout = 60s    # Standby disconnects if no message for 60s
wal_receiver_status_interval = 10s # How often standby sends status to primary
```

Network Bandwidth

```
# Estimate replication bandwidth (WAL generation rate)
# On primary:
psql -c "SELECT pg_wal_lsn_diff(pg_current_wal_insert_lsn(), '0/0') /
        EXTRACT(EPOCH FROM (now() - pg_postmaster_start_time()))
        AS bytes_per_second;"

# With wal_compression = on, actual network usage is typically 30-70% lower
```

Standby Apply Performance

```
# On standby: tune apply performance
wal_receiver_status_interval = 10s # Status update frequency
recovery_min_apply_delay = 0       # Set > 0 for intentional delayed replica
```

Interview Questions

Q1: Walk me through setting up streaming replication from scratch.

A: (1) Create replication user on primary. (2) Configure `wal_level=replica` , `max_wal_senders` , `max_replication_slots` in `postgresql.conf`. (3) Add `host replication replicator <standby-ip> scram-sha-256` to `pg_hba.conf`. (4) Reload primary. (5) Run `pg_basebackup --wal-method=stream --write-recovery-conf` on standby. (6) Set `primary_conninfo` in `postgresql.auto.conf` on standby. (7) Create `standby.signal` file. (8) Start standby and verify with `SELECT pg_is_in_recovery()` and `pg_stat_wal_receiver` .

Q2: What is the difference between `recovery.conf` and `postgresql.auto.conf` for standby configuration?

A: In PostgreSQL 11 and earlier, standby settings were in `recovery.conf` (separate file). In PostgreSQL 12+, `recovery.conf` was abolished. Standby settings go in `postgresql.conf` or `postgresql.auto.conf`, and a `standby.signal` file in the data directory triggers standby mode. Using the old `recovery.conf` in PG12+ is silently ignored, which is a common migration mistake.

Q3: What does `--wal-method=stream` do in `pg_basebackup`?

A: It causes `pg_basebackup` to open a second connection to the primary and stream WAL in parallel while copying data files. This ensures WAL generated during the backup is captured, preventing the gap between backup start and data file copy. Without it (using `fetch`), WAL from during the backup is retrieved at the end, requiring the primary to retain it all — which may fail if the backup takes a long time.

Q4: How do you prevent the standby from lagging too far and being unable to reconnect?

A: Two mechanisms: (1) `wal_keep_size` keeps a minimum amount of WAL regardless of slots, acting as a safety net. (2) Physical replication slots guarantee WAL retention until the standby consumes it. Best practice is to use both. Monitor slot lag and alert before disk fills.

Q5: What is `hot_standby_feedback` and when should you enable it?

A: When `hot_standby_feedback = on`, the standby periodically sends its oldest active transaction ID to the primary. The primary uses this to avoid vacuuming rows that the standby's long-running queries might still need. Without it, vacuum on the primary can cause "query conflict with recovery" cancellations on the standby. Enable it when standby has long-running queries. Downside: it can delay vacuum/autovacuum on the primary.

Q6: How do you check if a server is a primary or standby?

A: `SELECT pg_is_in_recovery();` — returns `true` on standby, `false` on primary. Also check: `SELECT * FROM pg_stat_wal_receiver;` (non-empty on standby), or check for `standby.signal` file in data directory.

Q7: What happens when a standby's `primary_slot_name` slot doesn't exist on the primary?

A: The standby will fail to start with an error like `replication slot "standby1_slot" does not exist`. The slot must be created on the primary before the standby starts, or `primary_slot_name` must be removed from standby config.

Q8: How do you re-sync a standby that has fallen too far behind?

A: (1) Stop the standby. (2) Remove data directory. (3) Re-run `pg_basebackup`. (4) Reconfigure `primary_conninfo`. (5) Restart. Alternatively, if the standby was a former primary and just needs to catch up a small divergence, use `pg_rewind` which is much faster.

Exercises and Solutions

Exercise 1: Full Setup Script

Write a shell script that automates primary configuration for streaming replication.

```
#!/bin/bash
# setup_primary.sh
# Run as postgres user on primary server
```

```

PGDATA=/var/lib/postgresql/16/main
REPL_USER=replicator
REPL_PASSWORD="$(openssl rand -base64 24)"
STANDBY_IP=192.168.1.101

echo "Creating replication user..."
psql -c "CREATE ROLE $REPL_USER WITH REPLICATION LOGIN PASSWORD '$REPL_PASSWORD';"

echo "Configuring postgresql.conf..."
cat >> $PGDATA/postgresql.conf << EOF

# Replication configuration (added by setup_primary.sh)
wal_level = replica
max_wal_senders = 10
max_replication_slots = 10
wal_keep_size = 1GB
wal_compression = on
wal_log_hints = on
hot_standby_feedback = on
track_commit_timestamp = on
EOF

echo "Configuring pg_hba.conf..."
echo "host replication $REPL_USER ${STANDBY_IP}/32 scram-sha-256" >>
$PGDATA/pg_hba.conf

echo "Creating replication slot..."
psql -c "SELECT pg_create_physical_replication_slot('standby1_slot', true);"

echo "Reloading PostgreSQL..."
pg_ctl reload -D $PGDATA

echo ""
echo "Primary setup complete!"
echo "Replication password: $REPL_PASSWORD"
echo "Run pg_basebackup on standby with user: $REPL_USER"

```

Exercise 2: Monitor Replication Health

Create a comprehensive monitoring query that shows replication health as a dashboard.

```

WITH replication_info AS (
    SELECT
        application_name,
        client_addr::text,
        state,
        sync_state,
        COALESCE(replay_lag, INTERVAL '0') AS lag,
        pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn) AS lag_bytes
    FROM pg_stat_replication
),

```

```

slot_info AS (
    SELECT
        slot_name,
        active,
        pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn) AS slot_lag_bytes
    FROM pg_replication_slots
    WHERE slot_type = 'physical'
)
SELECT
    'Standby Count' AS metric,
    COUNT(*)::text AS value
FROM replication_info
UNION ALL
SELECT 'Max Lag Seconds', MAX(EXTRACT(EPOCH FROM lag))::text
FROM replication_info
UNION ALL
SELECT 'Sync Standbys', COUNT(*)::text
FROM replication_info WHERE sync_state = 'sync'
UNION ALL
SELECT 'Inactive Slots', COUNT(*)::text
FROM slot_info WHERE active = false
UNION ALL
SELECT 'Max Slot Lag', pg_size_pretty(MAX(slot_lag_bytes))
FROM slot_info;

```

Cross-References

- [01 replication overview.md](#) — Replication concepts
- [04 synchronous replication.md](#) — Synchronous mode configuration
- [05 failover procedures.md](#) — Promoting standby to primary
- [06 patroni.md](#) — Automated HA with Patroni
- [09 ha architecture patterns.md](#) — Production HA designs